

Escalabilidade da difusão viscosa no NPAD

Eugenio Vitor Lopes dos Santos

DCA3703, Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

Avaliar a escalabilidade do código de Navier–Stokes em um nó do NPAD. Identificar gargalos e comparar implementações otimizadas, discutindo escalabilidade geral, fraca e forte.

2 Fundamentação teórica

A equação $\partial u / \partial t = \nu \nabla^2 u$ é discretizada por Euler explícito e diferenças centrais, formando o stencil de cinco pontos:

$$u_{i,j}^{k+1} = u_{i,j}^k + r(u_{i-1,j}^k + u_{i+1,j}^k + u_{i,j-1}^k + u_{i,j+1}^k - 4u_{i,j}^k), \quad r = \nu \Delta t / h^2.$$

Com $\nu = 0,1$, $\Delta t = 0,2$ e $h = 1$, obtém-se $r = 0,02$, dentro do limite de estabilidade bidimensional. Dois campos em `double` separam leitura e escrita. As bordas ficam fixas. Os campos nulo e constante devem permanecer invariantes, enquanto a perturbação gaussiana se difunde.

2.1 Distribuição e sincronização

O escalonamento `static` distribui blocos contíguos de linhas, adequado ao custo uniforme das células, em região paralela única nos 500 passos. Cada passo depende do anterior (RAW) e exige barreira após o stencil; a troca em `single` acrescenta segunda barreira, e ponteiros privados reduzem a troca a operação local. O bloco conservado por thread favorece a localidade temporal; o laço interno em `j` contíguo favorece a espacial.

2.2 SIMD e inicialização paralela

A diretiva `omp simd` sugere o processamento vetorial do laço interno, mas o compilador pode vetorizar sozinho, sem garantia de ganho. A inicialização paralela distribui a escrita dos dois campos; sob *first touch* em NUMA, cada página se aproxima da thread que a escreveu primeiro.

2.3 Indicadores de escalabilidade

Para trabalho fixo W , o speedup e a eficiência são

$$S(p, W) = \frac{T(1, W)}{T(p, W)}, \quad E(p, W) = \frac{S(p, W)}{p}.$$

- **Desempenho.** Tempo de execução da aplicação.
- **Eficiência.** Trabalho realizado com os recursos disponíveis para a execução.

- **Escalabilidade.** Capacidade de obter melhor desempenho à medida que mais recursos computacionais são utilizados.
- **Lei de Amdahl.** A parte do código que permanece sequencial limita o speedup máximo, por mais recursos que se adicionem.
- **Lei de Gustafson.** Quando o problema cresce com a quantidade de processadores, a aplicação pode continuar apresentando ganhos significativos.
- **Speedup.** Razão entre o tempo da referência e o tempo paralelo. Neste trabalho a referência é a própria estratégia com uma thread, pois o job não inclui versão sequencial.
- **Escalabilidade forte.** Problema fixo, com aumento apenas do número de processadores.
- **Escalabilidade fraca.** O problema cresce proporcionalmente aos processadores, mantendo aproximadamente a mesma carga por processador.
- **Fortemente escalável.** Mantém a eficiência ao aumentar os recursos sem aumentar o problema.
- **Fracamente escalável.** Mantém a eficiência quando o problema cresce menos que os recursos.

O tempo paralelo soma a parcela útil dividida por p ao overhead da paralelização: criação do conjunto de threads, distribuição e sincronização. Quando o problema cresce e o overhead cresce mais devagar que o trabalho útil, speedup e eficiência sobem; no caso contrário, a eficiência cai. Em trabalho fixo, a eficiência costuma cair com p ; com p fixo, costuma subir com o tamanho do problema. Na matriz deste job, cada linha fixa testa o caso forte e a diagonal proporcional testa o caso fraco usual.

3 Experimento

O job 2106246 rodou no nó `r2n04.npad.internal` (dois AMD EPYC 7713, oito domínios NUMA, 256 CPUs lógicas), na partição `amd-512`, com um processo, 64 CPUs e uso exclusivo.

A compilação usou `-std=c11 -O2 -Wall -Wextra -Werror -lm`, com `-fopenmp` e a instrumentação manual do PaScal. A flag `-ffp-contract=off` desliga a contração em FMA. A medição usa `omp_get_wtime()`. A evolução cronometrada cobre stencil, região OpenMP e sincronização; o total vai da alocação ao fim da evolução, sem diagnóstico, exportação e liberação. O trabalho é $W = (N - 2)^2 M$, com $M = 500$ passos em modo gaussiano, em dois campos de cerca de $16N^2$ bytes.

A matriz cruza sete cargas com sete contagens de threads nas quatro versões, com uma repetição por configuração: 49 execuções por versão. As cargas dobram a cada ponto, com erro de arredondamento abaixo de 0,022 % (Tab. 2). O código instrumenta inicialização (região 1, com first touch) e evolução (região 2); as figuras usam a região do programa inteiro.

Estratégia	Alteração acumulada
Static com barreiras	Região persistente e troca de ponteiros compartilhados.
Ponteiros privados	Troca local dos ponteiros, com uma barreira por passo.
SIMD	Ponteiros privados e <code>omp simd</code> no laço interno.
Inicialização paralela	Mesmo stencil com SIMD e escrita inicial paralela dos campos.

Tabela 1: Estratégias comparadas e alterações na implementação.

p na diagonal	Alvo de células	N	Células efetivas	Efetivo/alvo
1	262144	514	262144	1,000000
2	524288	726	524176	0,999786
4	1048576	1026	1048576	1,000000
8	2097152	1450	2096704	0,999786
16	4194304	2050	4194304	1,000000
32	8388608	2898	8386816	0,999786
64	16777216	4098	16777216	1,000000

Tabela 2: Matriz de cargas do job. Cada N inclui as bordas; $N - 2$ é o lado interno.

4 Implementação

Todas as estratégias calculam o mesmo stencil e mantêm dois campos. Como na Tarefa 11, as versões com inicialização serial preenchem o primeiro campo e o copiam para o segundo com `memcpy`, fora do intervalo de evolução. A inicialização paralela substitui essas etapas pela escrita dos dois campos no laço distribuído. A comparação entre estratégias altera sincronização, vetorização ou inicialização, preservando a aritmética e o número de passos.

4.1 Static com barreiras e ponteiros privados

Na estratégia `static` com `single`, a barreira do `omp for` conclui as escritas antes da troca, e a barreira do `single` publica os novos ponteiros antes do próximo passo:

```

1 #pragma omp single
2 { double *tmp = u; u = v; v = tmp; }
```

Listing 1: Troca dos ponteiros compartilhados após o stencil.

Com ponteiros privados, cada thread inicializa $a = u$ e $b = v$ dentro da região paralela. Os campos continuam compartilhados, mas a troca passa a ser local:

```

1 double *tmp = a; a = b; b = tmp;
```

Listing 2: Troca local após a barreira do laço espacial.

A barreira do `omp for` permanece obrigatória pela dependência entre passos. Apenas a sincronização associada à troca compartilhada é removida. Com distribuição `static`, cada thread recebe

um bloco contíguo de linhas; em p alto, o número de linhas por thread cai e o custo relativo das barreiras cresce.

4.2 SIMD e inicialização paralela

A estratégia SIMD mantém os ponteiros privados e acrescenta uma diretiva ao laço interno:

```
1 #pragma omp simd
2 for (int j = 1; j < n-1; ++j) {
3     /* Mesmo stencil. */
4 }
```

Listing 3: Vetorização explícita do laço de células.

A estratégia com inicialização paralela mantém esse cálculo e distribui também as linhas na escrita inicial:

```
1 #pragma omp parallel for schedule(static) default(none) shared(u,v,n,mode)
```

Listing 4: Distribuição da inicialização dos dois campos.

Essa alteração reduz o trabalho serial anterior à evolução e permite investigar a colocação inicial das páginas.

5 Resultados

A validação local passou em 195 comparações, com campos completos, passos pares, ímpares e zero, e execuções com uma, duas e quatro threads. Os campos nulo e constante ficaram invariantes. As figuras do Viewer classificam a região do programa inteiro em escalas normalizadas: ordenam as configurações entre si, sem exibir segundos. Em todos os diagramas, as linhas indicam threads (p), as colunas indicam cargas crescentes ($i1-i7$) e os tons indicam a posição relativa na escala de cada figura. O job do PaScal tem uma repetição por configuração. Sem mediana e sem dispersão, diferença pequena entre versões não ordena nada.

5.1 Desempenho

Tons claros marcam as combinações relativamente melhores. Nas quatro versões, o melhor relativo aparece em p intermediário. Nos extremos, pesam mecanismos distintos: com poucas threads, quase todo o trabalho recai sobre cada thread; com muitas threads e carga pequena, a fatia por thread encolhe enquanto o custo de criação do conjunto e de cruzar as barreiras a cada passo permanece. A distribuição static reparte blocos contíguos de linhas, de modo que a fração de linhas por thread cai com p e o peso relativo da sincronização cresce. As quatro versões dividem o mesmo trabalho da mesma forma; as diferenças entre elas atuam sobre essa estrutura comum dominante. Por isso os quatro mapas têm a mesma forma, e nenhuma versão se destaca na comparação visual.

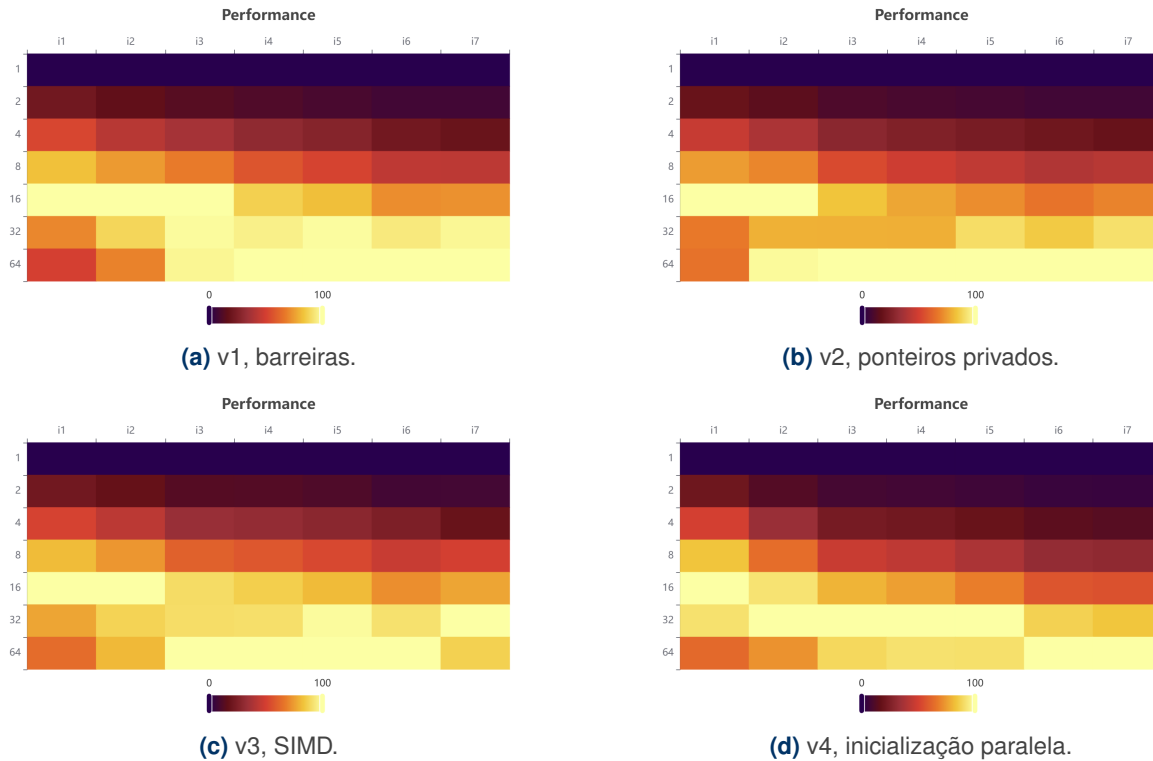


Figura 1: Desempenho do programa inteiro por versão. Linhas: threads. Colunas $i1-i7$: cargas crescentes.

5.2 Speedup

O speedup do Viewer é próprio de cada versão, com referência em uma thread. Tons quentes marcam speedup maior dentro de cada figura. O crescimento acompanha o ideal linear apenas no início; com p maior, o overhead de sincronização por passo cresce junto e afasta a curva do ideal. Nas cargas grandes, o mesmo overhead representa fração menor do trabalho total, e o speedup se sustenta melhor. Ler cada coluna fixa equivale ao teste de escalabilidade forte.

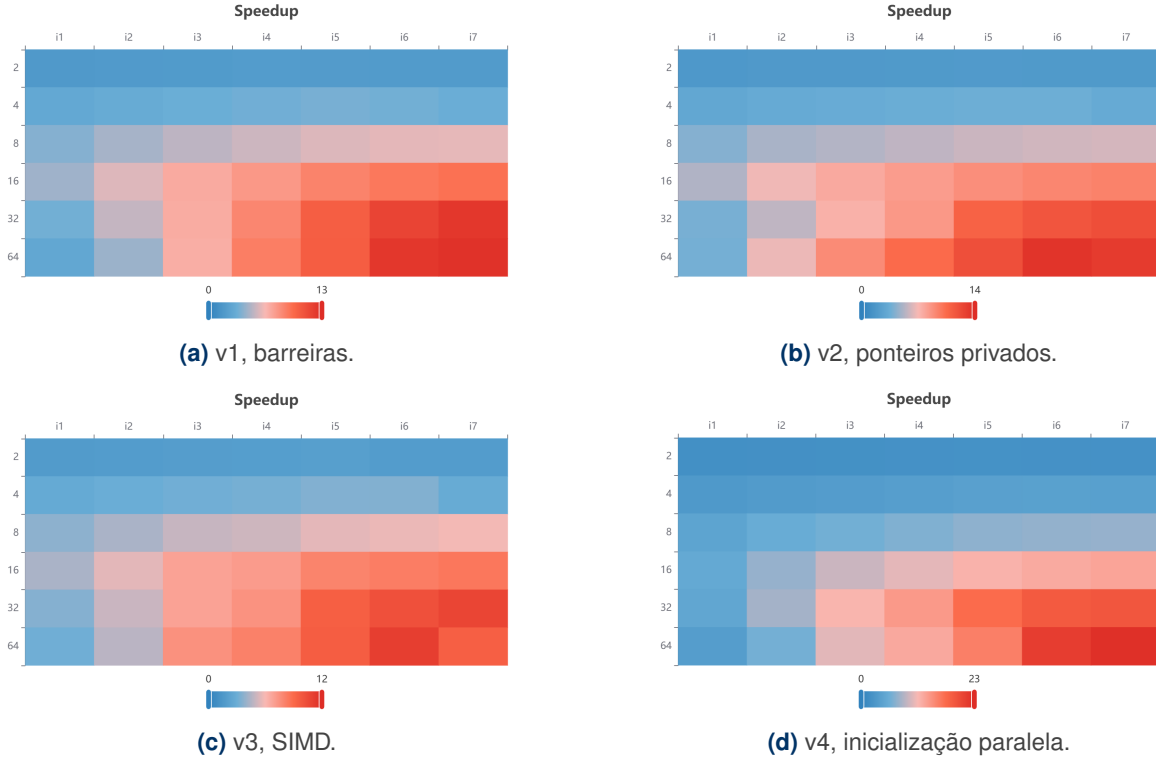


Figura 2: Speedup próprio por versão. Linhas: threads. Colunas $i1-i7$: cargas crescentes.

5.3 Eficiência

A eficiência divide o speedup por p . Tons escuros marcam eficiência maior. Em trabalho fixo, a eficiência cai com p em todas as versões e cargas: cada thread adicional acrescenta sincronização sem acrescentar trabalho. A queda acelera nas cargas pequenas, onde cada thread recebe poucas linhas e a parcela de coordenação domina a parcela útil.

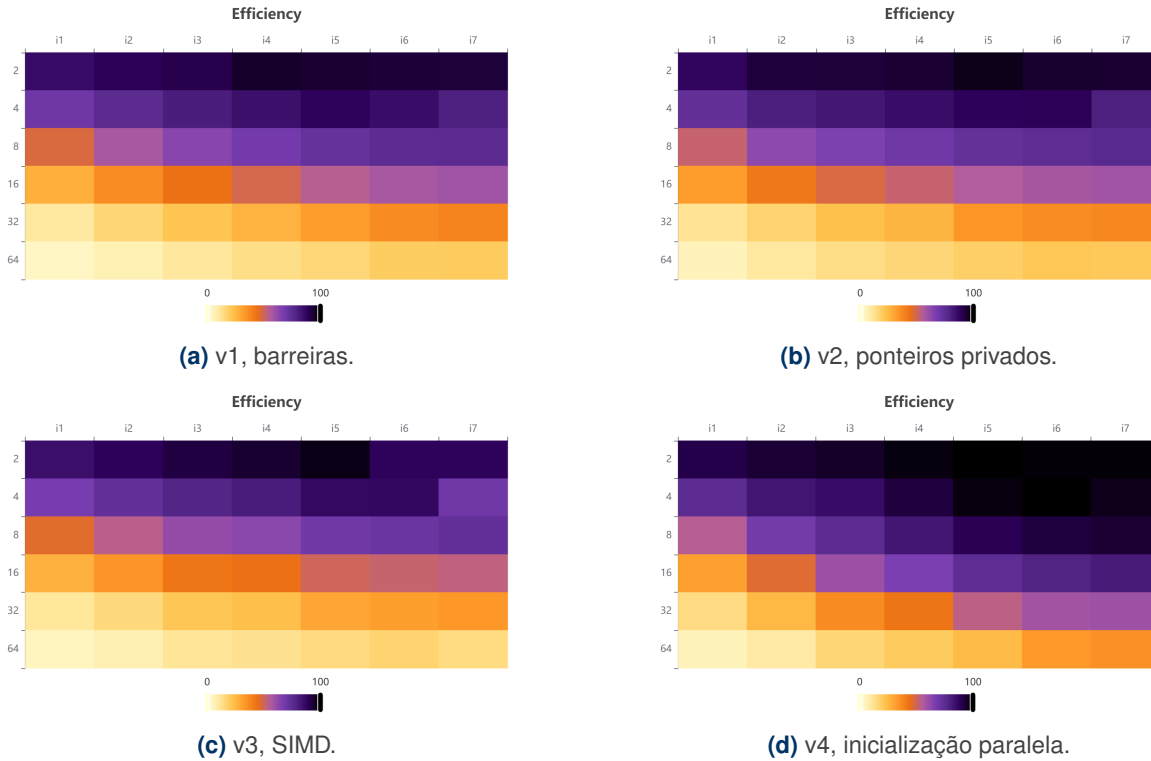


Figura 3: Eficiência por versão. Linhas: threads. Colunas $i1-i7$: cargas crescentes.

5.4 Escalabilidade forte

Cada coluna fixa a carga e varia as threads. O Viewer pinta quase tudo em tons de perda, piores em $p = 32$ e $p = 64$. A dependência temporal RAW entre passos exige uma barreira por passo em qualquer versão: são 500 sincronizações por execução, um custo de coordenação que não diminui quando p cresce. Com a carga fixa, cada thread recebe uma fatia cada vez mais fina do mesmo trabalho, e a coordenação passa a dominar. Nas fronteiras entre blocos, linhas de threads vizinhas podem dividir a mesma linha de cache e gerar tráfego de coerência, um mecanismo que cresce com p , mas os dados não o isolam. A célula isolada em $p = 2$ e carga média destoa do entorno; com apenas uma repetição, indica variação, não ganho.

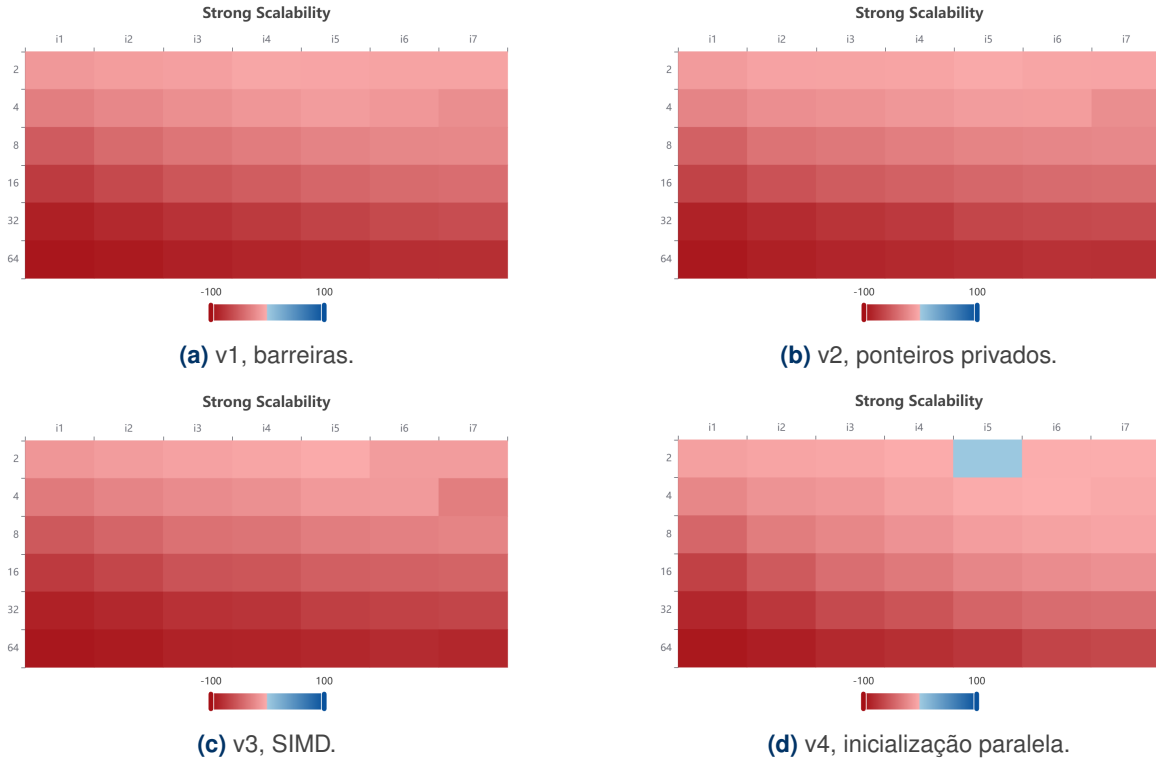


Figura 4: Escalabilidade forte do programa inteiro. Linhas: threads. Colunas $i1-i7$: cargas fixas por coluna.

5.5 Escalabilidade fraca

A diagonal mantém a carga por thread quase constante. Há escalabilidade fraca quando a eficiência se preserva ao aumentar problema e threads na mesma proporção. O Viewer pinta a diagonal na mesma faixa de perda das outras células, pior em $p = 64$: nessa faixa, dobrar carga e threads não preservou a eficiência. Duas ressalvas limitam a leitura. Primeiro, a região do programa inteiro inclui a parcela serial de alocação e inicialização, que impõe um teto à maneira de Amdahl sobre qualquer indicador medido no total. Segundo, a célula isolada em $p = 2$ reaparece aqui e indica variação, não ganho. O teste usual de carga constante por thread exige repetições e a região de evolução separada.

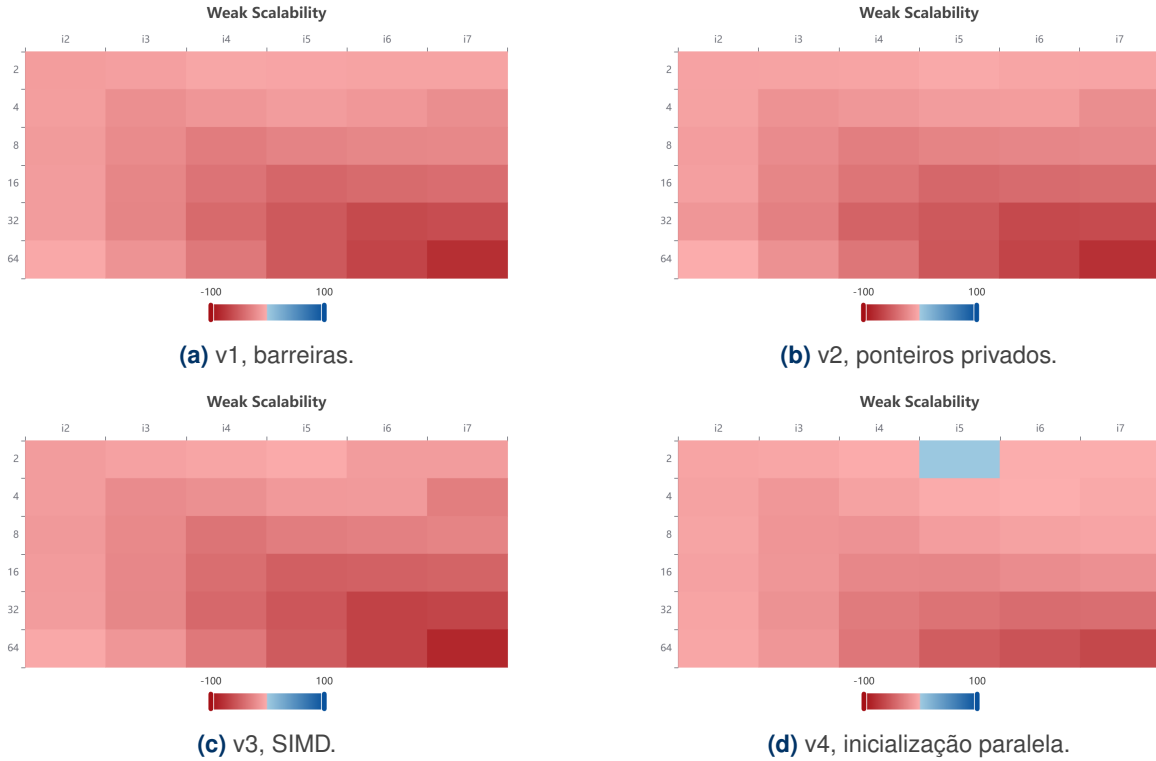
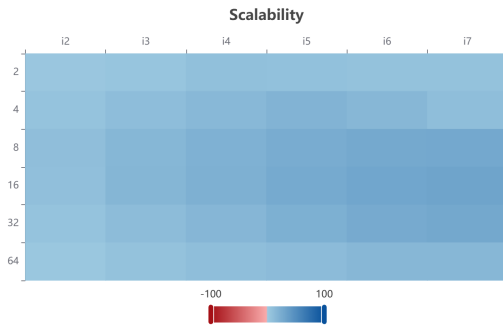


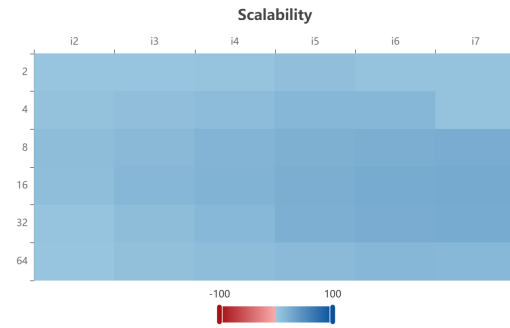
Figura 5: Escalabilidade fraca do programa inteiro. Linhas: threads. Colunas $i2-i7$: cargas da diagonal.

5.6 Escalabilidade geral

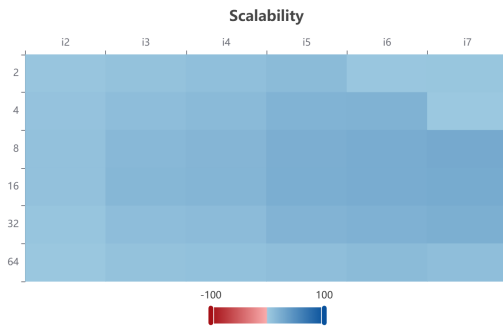
O diagrama geral combina trabalho e recursos. Um programa é escalável, no sentido formal, quando existe um ritmo de crescimento do problema que preserva a eficiência à medida que p aumenta. As quatro versões ficam em faixa intermediária, com o melhor relativo em $p = 8$ e $p = 16$ nas cargas médias e piora em $p = 64$. A eficiência se segura só no meio da faixa. Fora dali, overhead, barreiras e granularidade fina mandam. A figura cobre o programa inteiro, sem separar inicialização de evolução.



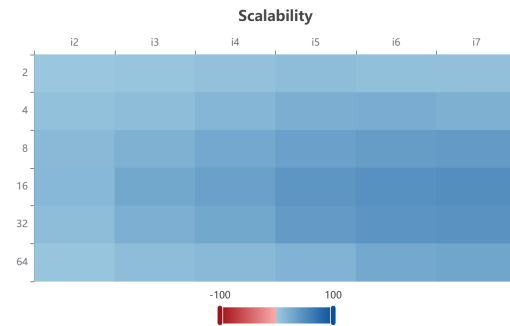
(a) v1, barreiras.



(b) v2, ponteiros privados.



(c) v3, SIMD.



(d) v4, inicialização paralela.

Figura 6: Escalabilidade geral do programa inteiro. Linhas: threads. Colunas $i2-i7$.

5.7 Comparação entre versões

Transição	O que muda	Efeito esperado	Observado
Barreiras → ponteiros privados	Sai a barreira do <code>single</code> ; ficam trabalho por thread, <code>static</code> e a barreira do <code>omp for (RAW)</code> .	Menos 500 sincronizações por execução.	Mesmo padrão nos mapas; sem evolução separada, o ganho não se distingue.
Ponteiros privados → SIMD	Entra <code>omp simd</code> no laço interno.	Vetorização explícita do laço de células.	Sem relatório de vetorização, sem efeito visível; o compilador pode já vetorizar sozinho.
SIMD → inicialização paralela	A escrita inicial dos dois campos passa ao laço distribuído.	Menor fração serial e <code>first touch</code> distribuído.	Sem evolução separada e sem dados de páginas, sem atribuição a NUMA.

Tabela 3: Transições entre estratégias: mudança, efeito esperado e observado.

A Código-fonte

Os arquivos `v1_barriers.c` (barreiras), `v2_private.c` (ponteiros privados), `v3_simd.c` (SIMD) e `v4_numa.c` (inicialização paralela) compartilham `main()`, medição com `omp_get_wtime()`, exportação e a aritmética do stencil. A atualização executada em cada célula é a mesma nas quatro versões:

```
1 size_t c = (size_t)i*n+j;
2 v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-n]+u[c+n]-4*u[c]);
```

Listing 5: Núcleo comum do stencil.

Da estratégia com barreiras para a de ponteiros privados, a troca compartilhada dá lugar à troca local:

```
1 #pragma omp single
2 { double *tmp = u; u = v; v = tmp; }
```

Listing 6: v1: troca em single, segunda barreira do passo.

```
1 double *a = u, *b = v; /* Ponteiros privados; campos compartilhados. */
2 /* Lacos espaciais e stencil identicos. */
3 double *tmp = a; a = b; b = tmp; /* Troca local, sem single. */
4 return steps % 2 ? v : u;
```

Listing 7: v2: ponteiros privados e troca local.

Da de ponteiros privados para SIMD, entra uma única linha no laço interno:

```
1 #pragma omp simd
```

Listing 8: v3: acréscimo sobre v2.

Da SIMD para inicialização paralela, a cópia serial dá lugar à escrita distribuída:

```
1 memcpy(v, u, count * sizeof(*v));
```

Listing 9: v3 e anteriores: inicialização serial.

```
1 #pragma omp parallel for schedule(static) default(none) shared(u,v,n,mode)
2 /* Mesmo preenchimento, nos dois campos. */
3 v[c] = u[c];
```

Listing 10: v4: inicialização distribuída.